KBCS: Karma-Based Congestion Signaling — System Methodology

1. Problem Statement

1.1 The Inter-CCA Fairness Problem

Modern networks carry traffic from applications that use different Congestion Control Algorithms (CCAs). These CCAs respond to congestion in fundamentally different ways:

CCA Category	Examples	How They React to Congestion
Loss-based	CUBIC, Reno, NewReno	Reduce sending rate when packets are dropped
Delay-based	Vegas, Copa	Reduce sending rate when round-trip time increases
Hybrid	Illinois, Westwood	Use both packet loss and delay signals
Model-based	BBR, BBRv2	Estimate available bandwidth; largely ignore drops

When these different CCAs share a bottleneck link, severe unfairness arises. Aggressive CCAs (CUBIC, BBR) consume the majority of bandwidth while cooperative CCAs (Vegas) get starved.

Quantified evidence (CCQM, Ma et al. 2026):

- CUBIC vs Vegas → Jain's Fairness Index (JFI) = **0.69** (severe unfairness)
- CUBIC vs BBR → JFI = **0.50** (near-total starvation of one flow)
- Same CCA competing → JFI ≈ **0.95–1.0** (fair)

1.2 Limitations of Existing Solutions

Traditional AQM (RED, CoDel, PIE): These drop packets based on queue depth or sojourn time, treating every flow identically. A cooperative Vegas flow and an aggressive CUBIC flow both face the same random drop probability. This worsens unfairness because Vegas, which is already self-throttling, gets penalized alongside the aggressive flow that is causing the congestion.

P4air (Turkovic & Kuipers, 2020): P4air uses P4 programmable switches to continuously monitor per-flow properties and fingerprint CCA behavior. It separates flows into groups and dynamically reallocates queues based on flow count. P4air proved that in-network fairness enforcement is viable at line-rate. However, two limitations exist:

1. **No recovery mechanism** — once a flow is classified as aggressive, it stays penalized indefinitely, leading to permanent starvation in long-lived flows.
2. **Static thresholds** — if network conditions change (flows join or leave), the fingerprinting parameters do not adapt.

CCQM (Ma et al., 2026): CCQM identifies the CCA type of each flow using a Binary Decision Tree by analyzing observable traffic features (inter-arrival times, throughput patterns). It then separates flows into dedicated queues by CCA category (loss-based, delay-based, hybrid,

model-based, short). This achieves 63% fairness improvement. However:

1. **Static queue boundaries** — the thresholds for classification and queue mapping are fixed at configuration time.
2. **No per-flow reputation** — within each queue, all flows of the same CCA type are treated equally regardless of whether an individual flow is behaving fairly or not.

PFQ (Wang et al., 2026): PFQ takes a proactive approach to buffer management. Instead of waiting for congestion to happen and then reacting (like RED), PFQ:

1. **Reserves buffer space** for potential new flow arrivals (proactive reservation).
2. **Recycles buffer** from flows that are under-utilizing their allocation and redistributes it to active flows.
3. **Periodically remaps** flows to physical queues based on forwarded packet counts, balancing load across available queues. This reduces packet drop rates by 74.3%. However, PFQ has no notion of reputation or fairness history — it treats all flows within their buffer allocation identically.

HINT (Sacco et al., 2023): HINT uses P4 switches to embed real-time telemetry data (processing delay, queue occupancy) directly into packet headers using In-Band Network Telemetry (INT). This data is extracted at the receiving end-host and used by the CCA to make better-informed rate decisions. HINT demonstrated that P4-driven telemetry can meaningfully improve congestion control accuracy with minimal overhead.

1.3 The KBCS Proposition

KBCS keeps the core system that no existing paper offers: **per-flow reputation tracking (karma) with explicit recovery**. On top of this core, we adopt specific proven techniques from the above papers to build a more robust system:

Source	What We Adopt	Why
P4air	Per-flow byte tracking in P4 registers as the fairness measurement foundation	Proven that P4-level per-flow accounting works at line-rate
CCQM	The concept of CCA-aware treatment — different CCA types need different enforcement	Validated that blind per-flow fairness is insufficient
PFQ	Proactive buffer reservation and buffer recycling from low-priority to high-priority flows	Addresses incast and improves utilization under dynamic flow counts
HINT	Telemetry-driven feedback loop — exporting switch state to a control plane for closed-loop optimization	Proves that P4-to-controller telemetry is lightweight and effective

What is NEW in KBCS (our contribution):

1. **Karma (reputation) tracking** — flows are judged on accumulated behavior over many windows, not just the current packet.
2. **Explicit recovery mechanism** — penalized flows get a formal second chance after sustained good behavior.
3. **Adaptive control loop** — the controller dynamically tunes fairness thresholds and penalty parameters based on live JFI feedback.
4. **Implicit CCA differentiation** — the karma mechanism naturally separates cooperators from aggressors without needing explicit CCA classification.

2.2 Why Two Tiers?

P4 is extremely fast but limited — it can count, compare, read/write registers, and make simple branch decisions. It cannot compute aggregate statistics (like JFI across all flows), cannot do floating-point math, and cannot implement learning algorithms. These require the Python control plane.

The data plane handles every packet at wire speed. The control plane runs every 2 seconds, reads aggregate state, makes optimization decisions, and pushes updated parameters back. This separation ensures that no per-packet decision ever waits for software.

2.3 Responsibility Division

Task	Where It Runs	Why There
Byte counting per flow	Data Plane	Must happen on every packet at line-rate
Karma update	Data Plane	Per-window decision, needs register read/write
Color assignment (GREEN/YELLOW/RED)	Data Plane	Simple threshold comparison per packet
Drop / ECN decision	Data Plane	Must be per-packet, immediate
Priority queue mapping	Data Plane	Direct hardware queue assignment
RED streak tracking and recovery	Data Plane	Per-flow register tracking
Telemetry export	Data Plane	Clone packets on state changes
Active flow counting	Control Plane	Requires aggregation across all flow registers
Fair bytes calculation	Control Plane	Division and multiplication beyond P4 capability
JFI computation	Control Plane	Requires sum-of-squares across all flows
Penalty/reward adjustment	Control Plane	Trend analysis over multiple epochs
PFQ buffer reservation update	Control Plane	Mathematical function of flow count

3. Data Plane Design — The Per-Packet Pipeline

Every TCP data packet (excluding SYN/FIN/RST) passes through the following stages inside the P4 switch.

3.1 Stage 1 — Flow Identification

Each flow is assigned a unique numeric ID via a P4 match-action table. The controller populates this table at experiment startup by mapping each host's source IP to a flow ID (1, 2, 3, ...). This is the same approach used by P4air.

3.2 Stage 2 — Byte Counting (Adopted from P4air)

For each identified flow, a P4 register accumulates the total bytes sent within the current measurement window. This is the fundamental measurement that all subsequent decisions depend on.

The measurement window is **15 milliseconds** (approximately 1.5× a typical RTT). This window duration balances two needs:

- Short enough to detect aggressive bursts quickly.
- Long enough to capture a complete send-and-feedback cycle (the flow sends data, gets an ACK, and adjusts its rate).

3.3 Stage 3 — Karma Update (KBCS Core Contribution)

At the end of each 15ms window, the switch evaluates the flow's behavior by comparing its byte count against the `fair_bytes` threshold (set dynamically by the controller).

If the flow exceeded `fair_bytes` (unfair behavior): A karma penalty is applied. The penalty scales proportionally to how far the flow exceeded its share:

- Exceeded by up to 1.5× → base penalty (e.g., 8 karma points)
- Exceeded by 4× → double penalty (16 points)
- Exceeded by 8× → quadruple penalty (32 points)

This proportional penalty is adopted from the principle behind CCQM's insight that different levels of aggressiveness need different treatment intensities.

If the flow stayed under `fair_bytes` (cooperative behavior): A karma reward is applied (e.g., 4 karma points). The reward is deliberately smaller than the penalty so that one window of aggression requires multiple windows of good behavior to recover from.

Karma Momentum (enhancement): If a flow's karma is dropping rapidly (more than 15 points lost compared to the previous window), an additional penalty is applied. This accelerates enforcement against flows that are ramping up aggressively.

Slow-Start Immunity: New flows (first 20 packets) are exempt from penalties. This prevents false penalization during TCP slow start, where flows naturally send increasing amounts of data while probing available bandwidth.

Idle Flow Recovery: If a flow has been silent for more than 500ms, it receives a small karma boost when it resumes. This prevents stale low-karma scores from lingering on flows that went quiet because they were starved by other flows.

3.4 Stage 4 — Color Zone Assignment (Inspired by CCQM's Classification)

Based on the accumulated karma score, the flow is placed into one of three zones. This is conceptually similar to how CCQM classifies flows into categories — but instead of classifying by CCA type (which requires complex feature extraction), KBCS classifies by behavioral reputation, which emerges naturally from the karma mechanism.

Zone	Karma Range	Meaning
GREEN	76–100	Consistently cooperative. Has been at or below fair share over multiple windows.

YELLOW	41–75	Moderate behavior. May be slightly aggressive or currently recovering.
RED	0–40	Consistently aggressive. Has been exceeding fair share repeatedly.

How different CCAs naturally end up in different zones:

CCA	Typical Behavior	Natural Karma Trajectory
Vegas	Self-throttles based on delay; stays well under fair share	Stays GREEN (high karma)
Reno	Moderate aggression; backs off quickly on loss	Stays GREEN/YELLOW
CUBIC	Ramps up aggressively until loss event	Oscillates YELLOW/RED
BBR	Probes bandwidth aggressively; ignores most drops	Drops to RED quickly

This means KBCS achieves CCA-aware treatment **without needing to explicitly classify CCAs**. The karma system naturally separates cooperative from aggressive flows. This is simpler than CCQM's Binary Decision Tree approach and avoids the problem of misclassification.

3.5 Stage 5 — Enforcement: Drop, ECN, and Buffer Management

This stage combines three enforcement mechanisms:

5a. Budget-Based Drops (KBCS Core)

Each color zone receives a different bandwidth budget within the window:

Zone	Budget	Drop Probability (if over budget)
GREEN	100% of fair_bytes	10% — gentle signal for loss-based CCAs
YELLOW	75% of fair_bytes	35% — moderate signal
RED	25% of fair_bytes	90% — aggressive signal needed for flows like BBR that ignore moderate drops

When a flow's bytes exceed its budget, a random number is generated and compared against the drop threshold. This is probabilistic (not deterministic) to avoid synchronization effects where all flows get dropped simultaneously.

5b. ECN Marking (Dual Signaling)

Packets that survive the drop decision are ECN-marked (the CE bit is set in the IP header). This provides a dual signaling strategy:

- Cooperative CCAs (CUBIC, Reno) that respond to ECN receive a gentle congestion signal and reduce their rate without losing data.
- Aggressive CCAs (BBR) that ignore ECN are forced to respond through the hard drop mechanism instead.

This dual approach is inspired by HINT's principle of providing rich congestion information to end-hosts — ECN-capable hosts can react to marks, while ECN-unaware hosts are still managed through drops.

5c. Proactive Buffer Reservation (Adopted from PFQ)

PFQ's key insight is that buffer space should be managed proactively, not just reactively. In KBCS, we adopt this through two mechanisms:

Buffer Reservation for New Flows: The controller maintains a "reserved buffer pool" that is not allocated to any specific flow. This is calculated based on the current number of active flows:

- When few flows are active → larger reservation (more room for new arrivals)
- When many flows are active → smaller reservation (most buffer is needed by existing flows)
- The reservation is capped at a maximum percentage of total buffer to prevent waste

This ensures that when new flows arrive (or existing flows suddenly burst, as in incast scenarios), there is always buffer space available rather than immediate drops.

Buffer Recycling from RED to GREEN Flows: In standard systems, all flows share the buffer equally. In KBCS, the budget restriction on RED flows (only 25% of fair share) means RED flows consume less buffer. The control plane accounts for this saved buffer space and redistributes it by increasing the effective `fair_bytes` for GREEN flows.

This is the KBCS adaptation of PFQ's buffer recycling mechanism: instead of recycling buffer from completed flows (PFQ's approach), we recycle buffer from penalized flows and give it to cooperative ones. This simultaneously:

- Punishes aggressive flows (less buffer budget)
- Rewards cooperative flows (more buffer budget)
- Maintains high link utilization (saved buffer is actually used, not wasted)

3.6 Stage 6 — RED Zone Recovery (KBCS Core Contribution)

This is one of the most important contributions of KBCS. No existing AQM — not RED, not CoDel, not P4air, not CCQM, not PFQ — provides a formal recovery path for penalized flows.

The Problem Without Recovery: A flow like BBR naturally probes bandwidth by periodically increasing its sending rate. Without recovery, BBR would get stuck in the RED zone permanently with 90% drops. Its throughput would collapse to near-zero indefinitely. This is not fairness — this is starvation. The link bandwidth that BBR would have used is wasted.

How Recovery Works: A dedicated register (`reg_red_streak`) counts how many consecutive 15ms windows a flow has been in the RED zone.

After **20 consecutive RED windows** (approximately 300ms):

1. The flow's karma is reset to 30 (low YELLOW zone)
2. The RED streak counter is reset to 0
3. The flow is reclassified as YELLOW

This gives the flow a "probation period." During this time:

- If the flow behaves cooperatively (stays under fair_bytes), its karma will gradually climb back through YELLOW toward GREEN.

- If the flow immediately becomes aggressive again, karma will drop right back to RED, and the 20-window countdown restarts.

Why 20 windows (300ms)? This is long enough that a truly aggressive flow has been meaningfully penalized (300ms of 90% drops is significant throughput loss), but short enough that the flow doesn't get permanently starved.

3.7 Stage 7 — Priority Queue Mapping

After the drop/ECN decision, surviving packets are assigned to hardware priority queues:

Zone	Queue Priority	Effect
GREEN	Priority 2 (highest)	Dequeued first; lowest latency
YELLOW	Priority 1 (medium)	Dequeued after GREEN packets
RED	Priority 0 (lowest)	Dequeued last; may face additional queue-based drops

BMv2 processes strict priority queues — GREEN packets are always served before YELLOW, and YELLOW before RED. This provides a second enforcement layer beyond drops.

3.8 Stage 8 — Telemetry Export (Adopted from HINT)

Borrowing from HINT's approach of embedding switch state for external consumption, KBCS exports per-flow telemetry to the control plane and visualization layer.

On specific events (color transitions, or periodically every 8th packet), the switch creates a **clone** of the packet. This clone carries:

- Flow ID
- Current karma score
- Current color zone
- Byte count
- Drop count

The clone is sent to a telemetry collector that writes the data to InfluxDB, which feeds the Grafana dashboard for real-time visualization and provides the data that the control plane uses for its optimization decisions.

This is not full INT (we do not embed telemetry into every transit packet header). Instead, we use P4's clone mechanism to generate dedicated telemetry packets only when state changes are detected. This is simpler and avoids the packet header overhead of full INT while still providing the closed-loop telemetry feedback that HINT demonstrated as valuable.

---

4. Control Plane Design — Q-Learning Based Adaptive Tuning

4.1 Why Reinforcement Learning for the Controller

The KBCS data plane has multiple interacting parameters: `fair_bytes`, `penalty_amt`, `reward_amt`, and the PFQ buffer reservation percentage. These parameters affect each other

— for example, increasing penalty makes flows drop to RED faster, which triggers more buffer recycling, which changes the effective fair share for GREEN flows. A simple rule-based controller (e.g., "if JFI < 0.70, increase penalty") cannot account for these interactions and may make suboptimal or conflicting adjustments.

Q-Learning is a model-free reinforcement learning technique that is well suited here because:

- It does not need a mathematical model of how the network behaves (which would be extremely complex to derive).
- It learns from direct observation: try an action, observe the result, and remember which actions led to good outcomes.
- Its Q-table is lightweight — our state and action spaces are deliberately small, so the table is only a few hundred entries.

4.2 The Control Loop

Every **2 seconds**, the controller executes one cycle:

1. **Observe** — Read P4 registers: per-flow packet counts, drop counts, ECN marks, forwarded bytes.
2. **Compute State** — Calculate JFI, link utilization, and active flow count. Discretize these into buckets to form the current state (see 4.3).
3. **Select Action** — Look up the current state in the Q-table to find the best action. With a small probability (exploration rate, starting at 10%), pick a random action instead to discover potentially better strategies.
4. **Execute Action** — Apply the selected action (e.g., increase penalty, decrease fair_bytes headroom). See 4.4 for the full action list.
5. **Update fair_bytes** — Always recalculate the base fair share from the current active flow count: `fair_bytes = (link_rate × window_duration) / active_flows × headroom_factor`.
6. **Update PFQ reservation** — Adjust buffer reservation percentage based on flow count.
7. **Write** — Push all updated parameters to P4 registers.
8. **Wait** — Let the network run for one epoch (2 seconds) under the new parameters.
9. **Measure Reward** — After the epoch, read fresh telemetry and calculate the reward (how much JFI and utilization improved or worsened).
10. **Learn** — Update the Q-table entry for the (previous state, action taken) pair using the standard Q-learning update rule.
11. **Log** — Export the epoch summary to InfluxDB for Grafana visualization.

4.3 State Space

The controller observes three metrics and discretizes each into buckets to form a compact state representation:

Metric	Buckets	Rationale
Jain's Fairness Index	< 0.70, 0.70–0.85, 0.85–0.95, > 0.95	Captures severity of unfairness
Link Utilization	< 30%, 30–60%, 60–80%, > 80%	Prevents over-penalization that kills utilization
Active Flow Count	1–4, 5–8, 9–16, > 16	Different flow counts need different parameter scales

Total state space = 4 × 4 × 4 = **64 states**. This is small enough for Q-learning to converge quickly (within a few minutes of traffic) while still capturing the key network conditions.

4.4 Action Space

Each action adjusts one parameter by a small, bounded amount:

Action	What It Does	When It Helps
`increase_penalty`	Penalty +2 (max 30)	JFI is low — need to punish aggressors harder
`decrease_penalty`	Penalty −1 (min 5)	JFI is high — can relax to improve utilization
`increase_reward`	Reward +1 (max 15)	Cooperative flows are recovering too slowly
`decrease_reward`	Reward −1 (min 2)	Flows are escaping RED zone too easily
`tighten_budget`	Reduce headroom factor by 0.1	Flows are exceeding fair share despite penalties
`loosen_budget`	Increase headroom factor by 0.1	Utilization is too low — flows need more room
`maintain`	No change	Current parameters are working well

Total action space = **7 actions**. Combined with 64 states, the Q-table has 64 × 7 = **448 entries** — trivially small for a Python dictionary.

4.5 Reward Function

After each epoch, the controller calculates a reward that captures what we care about — fairness, utilization, and absence of starvation:

Reward = 10 × (JFI improvement) + 3 × (utilization) − 5 × (starvation count)

Where:

- **JFI improvement** = current JFI − previous JFI. Positive if fairness got better.
- **Utilization** = current link utilization (0 to 1). Encourages the controller to not over-penalize.
- **Starvation count** = number of flows with throughput below 10% of fair share. Heavily penalizes solutions that starve any flow.

The weights (10, 3, 5) prioritize fairness above utilization but ensure the controller doesn't achieve "fairness" by throttling everyone equally to near-zero throughput.

4.6 Q-Learning Update

After each epoch, the Q-table entry for the state-action pair is updated using the standard rule:

Q(state, action) = Q(state, action) + α × [reward + γ × max Q(next_state, all actions) − Q(state, action)]

Where:

- **α (learning rate) = 0.1** — how much new experience overrides old knowledge
- **γ (discount factor) = 0.9** — how much future rewards matter relative to immediate ones

Over time, the Q-table converges: each state maps to the action that historically produced the best fairness-utilization trade-off.

4.7 Parameters Under Dynamic Control

Parameter	Default	Range	Controlled By
`fair_bytes`	Calculated from link rate and flow count	1500–15000	Direct calculation (always) + headroom from Q-learning
`penalty_amt`	8	5–30	Q-learning action selection
`reward_amt`	4	2–15	Q-learning action selection
`headroom_factor`	1.5	1.0–2.5	Q-learning action selection
Buffer reservation %	Based on flow count	5–30%	Direct calculation from flow count

Note that `fair_bytes` is always **mathematically calculated** from the link rate and flow count — Q-learning only adjusts the headroom factor that controls how much slack each flow gets above its strict fair share.

4.8 Why Q-Learning Works Here

Scenario: 4 flows → 2 flows

- Controller detects 2 active flows. `fair_bytes` is immediately recalculated (doubled).
- Next epoch: state changes from (JFI = 0.85, util = 0.70, flows = 1–4) to (JFI = 0.90, util = 0.50, flows = 1–4).
- Q-table suggests `loosen_budget` for this new state → headroom increases → remaining flows can use more bandwidth → utilization recovers.

Scenario: BBR dominating, JFI = 0.55

- State = (JFI < 0.70, util > 80%, flows = 1–4).
- Q-table has learned from past experience that `increase_penalty` works best in this state.
- Penalty increases → BBR karma degrades faster → BBR enters RED → 90% drops force throughput down → JFI recovers.
- On the next epoch, state moves to (JFI 0.70–0.85, ...) → Q-table may suggest `maintain` or another small adjustment.

Scenario: All flows are cooperative (all CUBIC, same rate)

- State = (JFI > 0.95, util > 80%, flows = 1–4).
- Q-table has learned that `maintain` or `decrease_penalty` works best here → controller does nothing or relaxes slightly.

4.9 Convergence and Safety Guarantees

- **Exploration decay:** The exploration rate starts at 10% and decays over time (every 50 epochs, halved). This ensures the controller explores in early epochs but stabilizes as the Q-table matures.
- **Bounded actions:** Every parameter has hard min/max limits. No Q-learning action can push a parameter outside its safe range.
- **Fallback:** If the Q-table is empty (first run), the controller falls back to conservative rule-based defaults (increase penalty if JFI < 0.70, decrease if JFI > 0.90) until enough epochs have been experienced to populate the table.

---

5. Multi-Switch Topology

5.1 Why Multiple Switches Matter

A single-switch dumbbell (hosts → one switch → server) is the standard testbed in most AQM papers including P4air and CCQM. While simple to set up, it only proves that the system works at a single congestion point. It does not answer critical questions like:

- What happens when a flow passes through two bottlenecks in sequence?
- Does karma enforcement at one switch interfere with enforcement at the next?
- Can independent KBCS instances coordinate without explicit communication?

In real data center and campus networks, traffic routinely passes through 2–3 switch hops. Proving KBCS works across multiple hops is essential for research credibility.

5.2 Why a Two-Tier Dumbbell (Not Leaf-Spine)

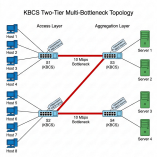
We deliberately chose a **two-tier dumbbell** over more complex topologies like leaf-spine for three reasons:

1. **It is the minimum topology that proves multi-hop behavior.** Two tiers of switches with bottleneck links between them is enough to show that KBCS handles cascading congestion. Going bigger (leaf-spine with 6+ switches) adds setup complexity without proving anything new about the karma mechanism itself.

2. **It is reproducible.** A 4-switch topology runs comfortably in Mininet on a standard VM. Leaf-spine with 6+ switches and 32+ hosts would require significant CPU/memory and introduce BMv2 software emulation artifacts that obscure the actual KBCS results.

3. **It has clear, identifiable bottlenecks.** The links between access and aggregation switches are rate-limited to 10 Mbps, creating well-defined congestion points. This makes it easy to measure and attribute fairness improvements to KBCS rather than to topology effects.

5.3 Topology Design



The topology consists of:

- **S1, S2 (Access Layer):** Each connects to a group of hosts running different CCAs (CUBIC, BBR, Vegas, etc.). These switches see the raw, unmanaged traffic from end hosts and perform the first round of KBCS karma evaluation.
- **S3, S4 (Aggregation Layer):** Connect to the destination servers. Traffic arriving here has already been filtered by S1/S2. These switches perform a second independent karma evaluation.
- **Bottleneck Links (10 Mbps):** The links between S1↔S3 and S2↔S4 are rate-limited, creating the congestion points where KBCS enforcement matters most.

5.4 Independent Karma Per Switch (Local Karma)

Each switch maintains its own set of karma registers. There is no karma sharing between switches.

Why this works:

- A flow passing through S1 → S3 is evaluated independently at each hop.
- If a flow is aggressive at S1, it accumulates low karma there and faces penalties (drops, low priority queue).
- At S3, it is evaluated fresh based on its behavior at that specific bottleneck.
- If still aggressive at S3, it also accumulates low karma there.

The result is that aggressive flows face **compounding enforcement** — penalties at every hop they traverse — while cooperative flows pass through all switches with high karma and high priority.

Why not share karma across switches? Sharing karma via INT headers or out-of-band protocols adds complexity, latency, and creates a dependency between switch instances. Since each bottleneck independently detects and penalizes aggressive behavior, the compounding effect is equivalent to shared karma but with zero coordination overhead. This also means the system scales to any number of switches without any inter-switch protocol.

---

6. Congestion Handling — Five-Phase Summary

KBCS handles congestion through five distinct phases, each building on the previous one:

```
┌──────────┐    ┌──────────────┐    ┌──────────────┐
│  DETECT  │───→│  ATTRIBUTE   │───→│  CLASSIFY    │
│          │    │              │    │              │
│ Count    │    │ Compare      │    │ Assign       │
│ bytes    │    │ against      │    │ GREEN /      │
│ per flow │    │ fair_bytes   │    │ YELLOW / RED │
│ per 15ms │    │              │    │ based on     │
│ window   │    │ Update karma │    │ karma score  │
└──────────┘    └──────────────┘    └──────────────┘
                                          │
     ┌────────────────────────────────────┘
     ↓
┌──────────────┐      ┌──────────────┐
```

```
│  ENFORCE      │──────→│  RECOVER      │
│               │       │               │
│ Drop / ECN    │       │ Track RED     │
│ per color     │       │ streak        │
│               │       │               │
│ PFQ buffer    │       │ After 20      │
│ reservation   │       │ RED windows:  │
│               │       │ boost karma   │
│ Priority      │       │ to YELLOW     │
│ queue assign  │       │               │
└───────────────┘       └───────────────┘
```

What makes this different from other AQMs:

AQM	Detection	Attribution	Classification	Enforcement	Recovery
RED	Queue depth	None (all flows equal)	None	Random drop	None
CoDel	Sojourn time	None	None	Scheduled drop	None
P4air	Per-flow bytes	Against BDP estimate	By CCA group	Queue reallocation	None
CCQM	Traffic features	By CCA classifier	By CCA category	Queue separation	None
PFQ	Buffer occupancy	By flow quota	None	Buffer restriction	None
KBCS	**Per-flow bytes**	**Against dynamic fair_bytes**	**By karma reputation**	**Drops + ECN + PFQ + priority**	**RED streak recovery**

---

7. Integration Map — What Comes from Where

This section clearly maps which technique originates from which paper and what KBCS adds on top.

7.1 Foundation: Per-Flow Byte Tracking (from P4air)

Original in P4air: Track per-flow byte counts in P4 registers. Compare against an estimated BDP (Bandwidth-Delay Product) to identify aggressive flows. Group flows and reallocate queues dynamically.

What KBCS adopts: The per-flow byte tracking using P4 registers. This is the measurement foundation.

What KBCS adds on top: Instead of comparing against a static BDP, we compare against a dynamically calculated `fair_bytes` that the controller adjusts based on live flow count and JFI. Instead of binary classification (aggressive/not), the byte comparison feeds into a cumulative karma score.

7.2 CCA-Aware Treatment (Inspired by CCQM)

Original in CCQM: Use a Binary Decision Tree to explicitly classify each flow's CCA type (loss-based, delay-based, hybrid, model-based). Route flows into separate dedicated queues by CCA type.

What KBCS adopts: The core insight that different CCA types need different treatment intensity. CCQM proved that blind equal treatment fails.

What KBCS adds on top: Instead of explicit classification (which can misclassify and requires feature extraction), KBCS achieves CCA-aware treatment implicitly through karma. Vegas naturally maintains high karma because it self-throttles. BBR naturally drops to low karma because it pushes aggressively. The differentiated enforcement per karma zone then automatically gives each CCA the appropriate signal strength without needing to know which CCA it is.

7.3 Proactive Buffer Management (Adopted from PFQ)

Original in PFQ: Three mechanisms — (1) proactively reserve buffer space for new flows, (2) recycle buffer from underutilizing flows to active flows, (3) periodically remap flows to physical queues for load balance.

What KBCS adopts:

1. **Buffer reservation** — The controller maintains a reserved buffer percentage that shrinks as more flows become active and grows as flows leave. This ensures new flow arrivals always have buffer space.
2. **Buffer recycling** — The bandwidth budget saved by restricting RED flows (75% reduction) is redistributed to GREEN flows through an increased effective `fair_bytes` for cooperative flows.

What KBCS adds on top: PFQ's recycling is flow-completion-based (recycle when a flow finishes). KBCS's recycling is reputation-based (recycle from penalized flows to rewarded flows continuously). This creates a direct incentive: behave well → get more resources. Behave badly → your resources go to others.

7.4 Telemetry-Driven Feedback (Inspired by HINT)

Original in HINT: P4 switches embed queue occupancy and processing delay into packet headers (full INT). End-hosts extract this data and use it for better CCA decisions.

What KBCS adopts: The principle of exporting real-time switch state for closed-loop optimization. We use P4's clone mechanism to generate telemetry packets carrying karma, byte counts, and drop counts.

What KBCS adds on top: Instead of feeding telemetry to end-hosts (HINT's approach), we feed it to our control plane which uses it to make parameter optimization decisions. This keeps the optimization within the network rather than relying on end-host cooperation.

---

8. Evaluation Methodology

8.1 Metrics

2	Explicit RED zone recovery	**KBCS original**	No prior AQM gives penalized flows a formal second chance
3	Implicit CCA differentiation via karma	**Inspired by CCQM**	Achieves CCA-aware treatment without explicit classification
4	Adaptive parameter tuning	**KBCS original**	Closed-loop JFI-based adjustment of penalty/reward
5	Proactive buffer reservation and recycling	**Adopted from PFQ**	Reputation-aware recycling (not just flow-completion-based)
6	Telemetry-driven control loop	**Inspired by HINT**	Switch-to-controller telemetry for parameter optimization
7	P4 line-rate implementation	**Foundation from P4air**	All per-packet logic runs in the data plane

References

1. Ma, H., Xu, D., Wang, X. (2026). "Congestion control algorithm-aware queue management." *Computer Networks* 276, 111975.
2. Wang, Y., Li, Q., Han, F., et al. (2026). "PFQ: A Proactive Fair Queueing Scheme Ensuring Fairness and High Utilization in Data Center Networks." *IEEE Transactions on Computers*.
3. Turkovic, B., Kuipers, F. (2020). "P4air: Increasing fairness among competing congestion control algorithms." *IEEE ICNP*.
4. Sacco, A., Angi, A., Esposito, F., Marchetto, G. (2023). "HINT: Supporting Congestion Control Decisions with P4-driven In-Band Network Telemetry." *IEEE HPSR*.
5. García-López, A., Kfoury, E.F., et al. (2025). "Real-Time Congestion Control Algorithm Identification with P4 Programmable Switches." *IEEE NOMS*.
6. Cardwell, N., et al. (2016). "BBR: Congestion-based congestion control." *ACM Queue*.
7. Ha, S., Rhee, I., Xu, L. (2008). "CUBIC: A new TCP-friendly high-speed TCP variant." *ACM SIGOPS Operating Systems Review*.